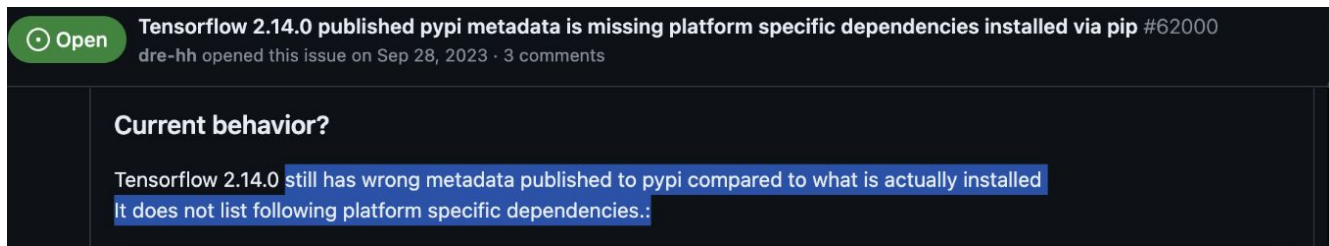




ArchDetective

Problem Description



- With code that involves different architectures such as ARM, x86, etc., it's crucial to ensure that the code is correctly labeled and targeted for the intended architecture.
- Failing to do so would result in issues like incompatibility, unnecessary resource consumption, reduced performance, system instability, security risks.
- There are no current solutions addressing this specific problem.

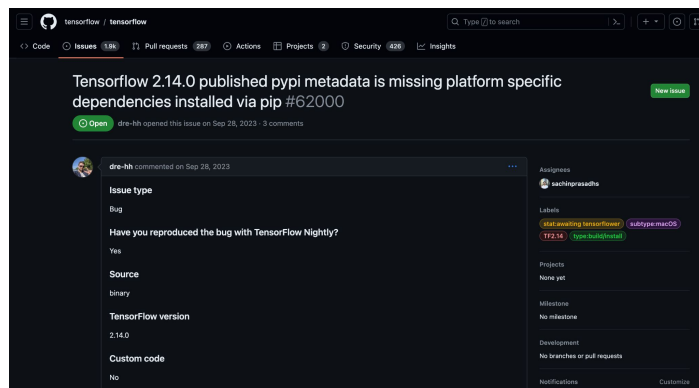
Proposed Solution - 1

- The first proposed solution was based on pip using binary wheels.
- Binary wheel: In complex archive format, it contains compiled C extension code.
- Pip retrieves distribution information by querying the URL https://pypi.org/simple/<package_name>.



Links for tensorflow

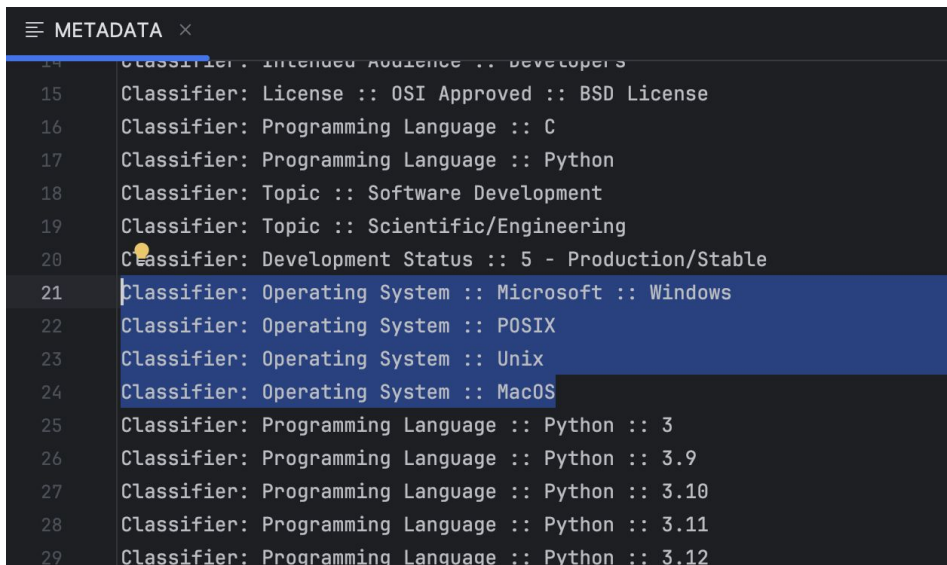
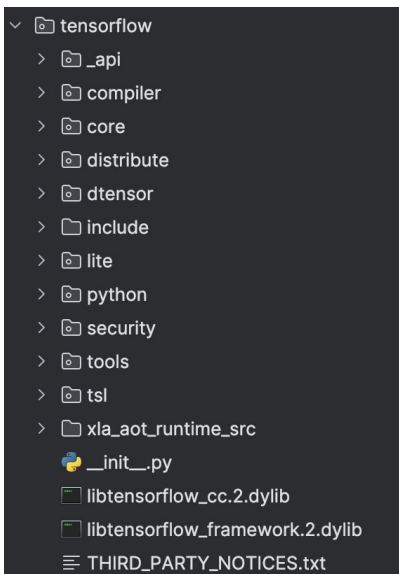
[tensorflow-0.12.0-cp27-cp27m-macosx_10_11_x86_64.whl](#)
[tensorflow-0.12.0-cp27-cp27mu-manylinux1_x86_64.whl](#)
[tensorflow-0.12.0-cp34-cp34m-manylinux1_x86_64.whl](#)
[tensorflow-0.12.0-cp35-cp35m-macosx_10_11_x86_64.whl](#)
[tensorflow-0.12.0-cp35-cp35m-manylinux1_x86_64.whl](#)
[tensorflow-0.12.0-cp35-cp35m-win_amd64.whl](#)
[tensorflow-0.12.1-cp27-cp27m-macosx_10_11_x86_64.whl](#)
[tensorflow-0.12.1-cp27-cp27mu-manylinux1_x86_64.whl](#)
[tensorflow-0.12.1-cp33-cp33m-manylinux1_x86_64.whl](#)
[tensorflow-0.12.1-cp34-cp34m-manylinux1_x86_64.whl](#)
[tensorflow-0.12.1-cp35-cp35m-macosx_10_11_x86_64.whl](#)
[tensorflow-0.12.1-cp35-cp35m-manylinux1_x86_64.whl](#)
[tensorflow-0.12.1-cp35-cp35m-win_amd64.whl](#)
[tensorflow-0.12.1-cp36-cp36m-macosx_10_11_x86_64.whl](#)
[tensorflow-0.12.1-cp36-cp36m-manylinux1_x86_64.whl](#)
[tensorflow-1.0.0-cp27-cp27m-macosx_10_11_x86_64.whl](#)
[tensorflow-1.0.0-cp27-cp27mu-manylinux1_x86_64.whl](#)
[tensorflow-1.0.0-cp33-cp33m-manylinux1_x86_64.whl](#)
[tensorflow-1.0.0-cp34-cp34m-manylinux1_x86_64.whl](#)
[tensorflow-1.0.0-cp35-cp35m-macosx_10_11_x86_64.whl](#)
[tensorflow-1.0.0-cp35-cp35m-manylinux1_x86_64.whl](#)
[tensorflow-1.0.0-cp35-cp35m-win_amd64.whl](#)
[tensorflow-1.0.0-cp36-cp36m-macosx_10_11_x86_64.whl](#)
[tensorflow-1.0.0-cp36-cp36m-manylinux1_x86_64.whl](#)
[tensorflow-1.0.1-cp27-cp27m-macosx_10_11_x86_64.whl](#)
[tensorflow-1.0.1-cp27-cp27mu-manylinux1_x86_64.whl](#)
[tensorflow-1.0.1-cp33-cp33m-macosx_10_11_x86_64.whl](#)
[tensorflow-1.0.1-cp33-cp33m-manylinux1_x86_64.whl](#)
[tensorflow-1.0.1-cp34-cp34m-macosx_10_11_x86_64.whl](#)
[tensorflow-1.0.1-cp34-cp34m-manylinux1_x86_64.whl](#)
[tensorflow-1.0.1-cp35-cp35m-macosx_10_11_x86_64.whl](#)
[tensorflow-1.0.1-cp35-cp35m-manylinux1_x86_64.whl](#)
[tensorflow-1.0.1-cp35-cp35m-win_amd64.whl](#)
[tensorflow-1.0.1-cp36-cp36m-manylinux1_x86_64.whl](#)
[tensorflow-1.1.0-cp27-cp27m-macosx_10_11_x86_64.whl](#)
[tensorflow-1.1.0-cp27-cp27mu-manylinux1_x86_64.whl](#)
[tensorflow-1.1.0-cp33-cp33m-macosx_10_11_x86_64.whl](#)
[tensorflow-1.1.0-cp33-cp33m-manylinux1_x86_64.whl](#)
[tensorflow-1.1.0-cp34-cp34m-macosx_10_11_x86_64.whl](#)
[tensorflow-1.1.0-cp34-cp34m-manylinux1_x86_64.whl](#)
[tensorflow-1.1.0-cp35-cp35m-macosx_10_11_x86_64.whl](#)



File Name Analysis: Identify patterns or keywords indicating architecture specificity.

Metadata Examination: Look for declarations or indications of the targeted architecture.

Binary Content Analysis: Identify patterns that signify the architecture for which the code is compiled.



Later...

Is it possible to implement this without having to download the package in local?

Apt.cache

Apt.cache typically refers to the package cache used by the Advanced Package Tool (APT) on Debian-based Linux distributions.

Apt.cache is a local collection of all the packages with their metadata that includes everything shown in the picture below on every package.

```
(kali@kali)-[~/Desktop]
$ python aptscan.py
['_class_', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
 '__getattr__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__',
 '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
 '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', '_changelog', '_pca
che', '_pkg', 'architecture', 'candidate', 'commit', 'essential', 'fullname', 'get_changelog',
'has_config_files', 'id', 'installed', 'installed_files', 'is_auto_installed', 'is_auto_removab
le', 'is_inst_broken', 'is_installed', 'is_now_broken', 'is_upgradable', 'mark_auto', 'mark_del
ete', 'mark_install', 'mark_keep', 'mark_upgrade', 'marked_delete', 'marked_downgrade', 'marked
_install', 'marked_keep', 'marked_reinstall', 'marked_upgrade', 'name', 'shortname', 'versions'
]
bound method Package.architecture of <Package: name='uuid' architecture='arm64' id:9549>>
```

Step 1

```
<bound method Package.architecture of <Package: name:'zaqar-tempest-plugin' architecture='arm64' id:122709>>
<bound method Package.architecture of <Package: name:'zarchive-tools' architecture='arm64' id:122710>>
<bound method Package.architecture of <Package: name:'zatacka' architecture='arm64' id:16761>>
<bound method Package.architecture of <Package: name:'zathura' architecture='arm64' id:27586>>
<bound method Package.architecture of <Package: name:'zathura-cb' architecture='arm64' id:122715>>
<bound method Package.architecture of <Package: name:'zathura-dev' architecture='arm64' id:122717>>
<bound method Package.architecture of <Package: name:'zathura-djvu' architecture='arm64' id:122714>>
<bound method Package.architecture of <Package: name:'zathura-pdf-poppler' architecture='arm64' id:122712>>
<bound method Package.architecture of <Package: name:'zathura-ps' architecture='arm64' id:122713>>
<bound method Package.architecture of <Package: name:'zaz' architecture='arm64' id:17014>>
<bound method Package.architecture of <Package: name:'zaz-data' architecture='arm64' id:122719>>
<bound method Package.architecture of <Package: name:'zbackup' architecture='arm64' id:122720>>
<bound method Package.architecture of <Package: name:'zbar-tools' architecture='arm64' id:107437>>
<bound method Package.architecture of <Package: name:'zbarcam-gtk' architecture='arm64' id:122721>>
<bound method Package.architecture of <Package: name:'zbarcam-qt' architecture='arm64' id:122722>>
```

Step 2

Checking it against **Filename** mentioned in the package metadata which represents the actual .deb file this package is mapped to download.

```
SHA1: 219902d23d0a1e2e0d62a830bb4f82d1ec39aa13
MD5sum: fc5ed8a20ce1861950c7ed3a5a615be0
Description: Real-time strategy game of ancient warfare (data files)
  0 A.D. (pronounced "zero ey-dee") is a free, open-source, cross-platform
  real-time strategy (RTS) game of ancient warfare. In short, it is a
  historically-based war/economy game that allows players to relive or rewrite
  the history of Western civilizations, focusing on the years between 500 B.C.
  and 500 A.D. The project is highly ambitious, involving state-of-the-art 3D
  graphics, detailed artwork, sound, and a flexible and powerful custom-built
  game engine.
.
This package contains the main data files required by 0 A.D.
Description-md5:
Homepage: https://play0ad.com/
Tag: role::app-data
Section: games
Priority: optional
Filename: pool/main/o/0ad-data/0ad-data_0.0.26-1_all.deb

(kali@kali)-[~/Desktop]
$ apt-cache show openssl | grep '^Filename:'
Filename: pool/main/o/openssl/openssl_3.1.5-1_arm64.deb

(kali@kali)-[~/Desktop]
```

Step 3

One more step, we can go to the file list mentioned by Debian and getting the info https://packages.debian.org/sid/<arch>/<package_name>/filelist and using **mimetypes** lib in python checking if there is any file that doesn't run on arm64 based on file extensions.

File list of package *openssl* in *sid* of architecture *arm64*

```
/etc/ssl/openssl.cnf
/usr/bin/c_rehash
/usr/bin/openssl
/usr/lib/ssl/cert.pem
/usr/lib/ssl/certs
/usr/lib/ssl/misc/CA.pl
/usr/lib/ssl/misc/tsget
/usr/lib/ssl/misc/tsget.pl
/usr/lib/ssl/openssl.cnf
/usr/lib/ssl/private
/usr/share/doc/openssl/FAQ.md
/usr/share/doc/openssl/HOWTO/certificates.txt.gz
/usr/share/doc/openssl/HOWTO/documenting-functions-and-macros.md.gz
/usr/share/doc/openssl/HOWTO/keys.txt
/usr/share/doc/openssl/NEWS.md.gz
/usr/share/doc/openssl/README-ENGINE5.md.gz
/usr/share/doc/openssl/README.Debian
/usr/share/doc/openssl/README.md.gz
/usr/share/doc/openssl/README.optimization
/usr/share/doc/openssl/changeLog.Debian.gz
/usr/share/doc/openssl/changeLog.gz
/usr/share/doc/openssl/copyright
/usr/share/doc/openssl/fingerprints.txt
/usr/share/doc/openssl/overrides/openssl
/usr/share/man/man1/CA.pl.1ssl.gz
/usr/share/man/man1/asn1parse.1ssl.gz
/usr/share/man/man1/c_rehash.1ssl.gz
/usr/share/man/man1/ca.1ssl.gz
/usr/share/man/man1/ciphers.1ssl.gz
/usr/share/man/man1/cmp.1ssl.gz
/usr/share/man/man1/cms.1ssl.gz
/usr/share/man/man1/crl.1ssl.gz
/usr/share/man/man1/crl2pkcs7.1ssl.gz
/usr/share/man/man1/dgst.1ssl.gz
/usr/share/man/man1/dhparam.1ssl.gz
/usr/share/man/man1/dsa.1ssl.gz
/usr/share/man/man1/dsaparam.1ssl.gz
```

What does this solve?

Debian Multiarch

Debian Multiarch is a feature in the Debian operating system that simplifies the process of cross-compiling.

It works by adding a foreign architecture to APT, then installing the foreign libraries right along-side your native ones. Allowing you to cross-compile just like you would regularly compile.

```
Architecture: arm64  
Multi-Arch: foreign  
Version: 3.1.4-2  
Depends: libc6 (≥ 2.34), libssl3 (≥ 3.0.9)  
Suggests: ca-certificates
```

Code Issues **5k** Pull requests **624** Actions Projects **2** Security **4** Insights

Cannot cross-compile aarch64-musl on x86_64 linux #46651

Closed asonix opened this issue on Dec 10, 2017 · 15 comments · Fixed by rust-lang/compiler-bultins#377 or #75877

asonix commented on Dec 10, 2017

Cross compiling fails with the linker being unable to find `__addtf3`, `__multf3` and `__subtf3`

code:

```
asonix@sonix-boxes-16-04:~/hello-world-rust$ cat src/main.rs
fn main() {
    println!("Hello, world!");
}
```

I'm compiling the default hello-world application, and it works fine for `aarch64-unknown-linux-gnu` in addition to working fine for `arm-unknown-linux-musleabi`. I installed musl the same way for both aarch64 and for armhf, so I'm not sure what the issue would be.

Furthermore, I can compile a simple hello-world application written in C with the musl standard library just fine.

Note: I have encountered this issue both in Arch Linux with the 4.14.4 kernel and Ubuntu 16.04 with the 4.4.0 kernel.

version info for the output below, although this happens with nightly and stable:

```
asonix@sonix-boxes-16-04:~/hello-world-rust$ rustc --version --verbose
rustc 1.22.1 (85e2e1c41 2017-11-22)
```

Assignees
No one assigned

Labels
C-bug **O-Arm** **O-musl** **T-compiler**

Projects
None yet

Milestone
No milestone

Development
Successfully merging a pull request may close this issue.
Add compiler-rt fallbacks on aarch64-musl
vignux/compiler-bultins
Update compiler-bultins
vignux/rust

Code Issues **5k** Pull requests **1.5k** Actions Projects **27** Security Insights

Cross compiling on Linux is untested, undocumented, and broken #90039

Closed tiran opened this issue on Nov 23, 2021 · 8 comments

tiran commented on Nov 23, 2021

BPO	45881
Nosy	@brettcannon @tiran @moreatl
PRs	→ bpo-45881: Use CC from env first for cross building (GH-29752) #29752 → [3.10] bpo-45881: Use CC from env first for cross building (GH-29752) (GH-29753) #29753 → [3.9] bpo-45881: Use CC from env first for cross building (GH-29752) (GH-29754) #29754 → bpo-45881: configure --with-freeze-module --with-build-python (GH-29835) #29835
Dependencies	- bpo-45886: Fix Program_freeze_module for cross compiling Python - bpo-45900: sysconfig --generate-posix-vars creates wrong file when cross compiling

Note: these values reflect the state of the issue at the time it was migrated and might not reflect the current state.

► Show more details

Assignees
No one assigned

Labels
3.9 **3.10** **3.11** **build** **docs** **tests**

Projects
None yet

Milestone
No milestone

Development
No branches or pull requests

Notifications
Customize
Subscribe

You're not receiving notifications from this thread.

Code **<>** Code ▾

Link to libgcc when statically linking musl #52091

Closed Amanieu wants to merge 1 commit into `rust-lang:master` from `Amanieu:aarch64_musl_libgcc`

Conversation **5** → **Commits** **1** **Checks** **0** **Files changed** **1** **+9 -0**

Amanieu commented on Jul 5, 2018

On some architectures (e.g. aarch64) musl depends on some libgcc functions (`__addtf3`, `__multf3`, `__subtf3`) for long double arithmetic that it uses internally. Unfortunately we don't provide these functions in compiler-bultins, so we instead need to get them from libgcc.

Fixes:
#46651
rust-lang/compiler-bultins#201
rust-lang/compiler-bultins#217

Reviewers
No reviews

Assignees
alexsricton

Labels
S-waiting-on-review

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.
None yet

rust-highfive assigned **varkor** on Jul 5, 2018

rust-highfive commented on Jul 5, 2018

r? @varkor
(rust_highfive has picked a reviewer for you, use r? to override)

Debian Multiarch

Pros

- Extremely easy when functional
- No need to compile libraries from source

Cons

- Limited to Debian.
- Only supports targeting your current distribution version.
- **Some packages conflict with foreign versions, preventing Multiarch from working properly.**
- **Ubuntu puts all non-x86 packages in a separate repository, requiring you to manually install the Ubuntu Ports repository.**
- **Root filesystem pollution.**

Limitations

One major limitation with this implementation of the scanner is that it only works for Debian-based Linux distributions.

No Apt - No scan

From my exploration, I found no way of using **File <filename>** and determine the filetype using binary without downloading the package first.

Next steps

Implement the step 3 of the scanner, where it looks for filetype using file extension.

Running manual scan and get more statistics.

Thank you

References

<https://dev.to/thebrokenrail/cross-compiling-is-way-too-difficult-how-can-we-fix-it-gnd>
<https://nixcademy.com/2024/01/30/cross-compilation-with-nix/>
https://www.reddit.com/r/rust/comments/ivivwj/help_me_understand_crosscompiling/
<https://news.ycombinator.com/item?id=18732832>
<https://github.com/python/cpython/issues/90039>
<https://github.com/rust-lang/rust/issues/46651>
<https://github.com/rust-lang/rust/pull/52091>
<https://github.com/mesonbuild/meson/issues/4694>
<https://wiki.debian.org/Multiarch/Issues>
<https://landley.net/writing/docs/cross-compiling.html>